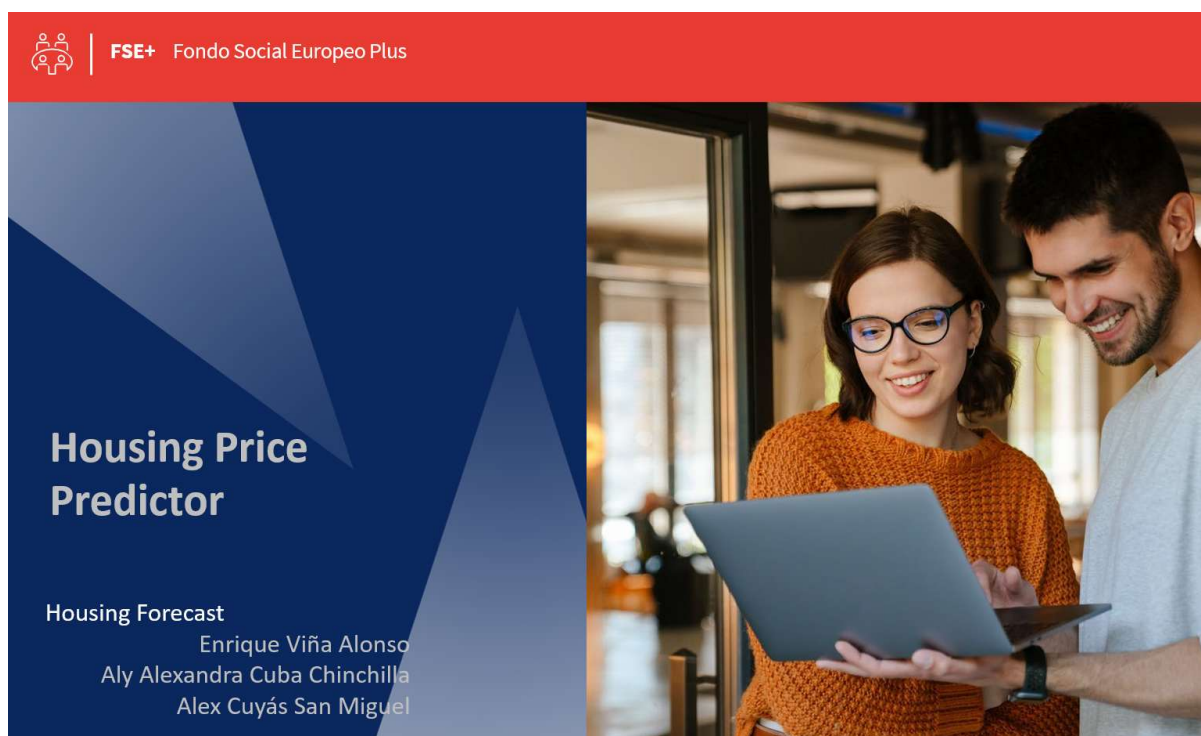


AI Course

Capstone Project Final Report



Título

dd/mm/aaaa

Housing Forecast

Alumno1
Alumno2
Alumno3

Content

1. Introduction

- 1.1. Background Information
- 1.2. Motivation and Objective
- 1.3. Members and Role Assignments
- 1.4. Schedule and Milestones

2. Project Execution

- 2.1. Data Acquisition
- 2.2. Training Methodology
- 2.3. Workflow
- 2.4. System Diagram

3. Results

- 3.1. Data Preprocessing
- 3.2. Exploratory Data Analysis (EDA)
- 3.3. Modeling
- 3.4. User Interface
- 3.5. Testing and Improvements

4. Projected Impact

- 4.1. Accomplishments and Benefits
- 4.2. Future Improvements

5. Team Member Review and Comment

6. Instructor Review and Comment

1. Introduction

1.1. Background Information

The project focuses on developing an artificial intelligence model designed to predict property prices in the municipality of Alicante.

To achieve this, we have accessed over 4,300 data points obtained through the Idealista API which offers a wealth of detailed information about the properties available in the municipality. This information will be used to create a price prediction model at local scale, which could be adapted for other regions in Spain, making it a scalable solution for the national scale.

This approach will help better understand the local market, leading to more informed and efficient decision-making in Alicante's estate sector. The model will offer valuable insights into market trends, ensuring that decisions by buyers, sellers and industry professionals are based on accurate data.

1.2. Motivation and Objective

The increase in house pricing has been a rising problem for the last few years and it has not shown signs of ending soon. We wanted to make a system that helps people find a suitable housing option according to both their physical and economical needs.

We decided on house pricing since it's something we all struggled before and had some basic understanding of the business and data we could look for. After discarding some pre existing datasets, we decided to create our own, acquiring the data with an API and cleaning the results so the data can be scalable to other cities.

The objective is to perform a housing price study and to find a model capable of predicting a fitting market price for a property, according to its characteristics and location, providing useful insights to both buyers and sellers.

1.3. Members and Role Assignment

Enrique, Aly and Alex met on the EOI AI Development course, where we learnt and practised everything we could need about machine learning and neural networks. We decided not to establish any defined roles, and instead work together in a peer collaborative environment.

1.4. Schedule and Milestones

Our first project meeting took place on November 18th, where we began planning the project using a bottom-up approach. The project was structured over four sessions, each lasting five hours.

The initial plan allocated:

- **Session 1:** Defining the project scope and creating an initial plan.
- **Session 2:** Data acquisition and processing.
- **Session 3:** Model selection and hyperparameter tuning.
- **Session 4:** Final documentation.

However, we anticipated that data acquisition and processing might take longer than five hours, while model selection and tuning might require less time. As a result, this schedule was designed to be a flexible framework rather than a rigid timeline.

2. Project Execution

2.1. Data Acquisition

Once the main research topic was defined, we conducted an information search to obtain a reliable and extensive dataset for making predictions. The data has been obtained through Idealista. We requested access to their API via email, and they provided the necessary credentials for API access, which includes a total of 100 calls. Along with the access data, Idealista also provided the information required to understand their database, enabling us to conduct the study properly and effectively.

2.2. Training Methodology

The training methodology for this project involves several steps to ensure the development of an accurate and reliable predictive model:

2.2.1 Model selection

We have used linear regression models for predicting the price of flats. This model is well-suited for this type of predictions as it assumes a linear relationship between the input features and the target variable, the price. We may also explore other variations of regression models such as Ridge regression or Lasso regression, Random Forest, bagging (bootstrap Aggregating) which can help handle multicollinearity and overfitting issues.

2.2.2 Data preparation

The first step in the process was Data Preprocessing, which occupied a significant portion of the time allocated for the project. This phase is crucial for preparing the data and ensuring its quality, as clean and well-structured data is essential for building an effective predictive model.

2.2.3 Training and validation

For the training phase of this project, We experiment with a variety of machine learning models to predict housing prices. The goal will be to find the best-performing model based on the dataset's features.

2.3. Workflow

We progressed through the project using blackboard and online workspaces like github and google docs alongside local machines for planning, testing, adding and improving the model.

2.4. System Design

The system design that will be used in this project has the following structure:

2.4.1 Data acquisition

Data obtained from various sources, such as APIs, databases and CSV files.

2.4.2 Data preprocessing

The acquired data is cleaned and prepared for use in the models. Includes removing null values, encoding categorical variables and normalizing features to ensure that models receive high-quality data.

2.4.3 Model training

Once the data is ready, different models, such as linear regression and Random Forest, are trained to find the best-performing model. The processed data is fed into the models to learn the relationships and patterns that predict housing prices.

2.4.4 Model evaluation.

The trained models are validated using metrics such as MAE, RMSE and R2. This step is essential to determine which model performs the best and provides the most accurate predictions.

2.4.5 Prediction

With the trained and validated model, predictions are made to estimate the housing prices. This step involves applying the model to new data to generate the desired results.

3. Results

3.1. Data Preprocessing

3.1.1 Deletion of duplicate entries

We check for any duplicate entries based on the 'propertyCode', which serves as the unique identifier for each property listing, and remove them if found.

3.1.2 Deletion of redundant columns

We remove unnecessary or duplicate columns from our dataset that don't add meaningful information or contribute to the analysis.

Some of the redundant columns include:

- externalReference
- numPhotos
- hasVideo
- country
- Others that do not contribute to the analysis.

```
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   price                                4345 non-null   float64
1   propertyType                         4345 non-null   object
2   size                                4345 non-null   float64
3   rooms                               4345 non-null   int64
4   bathrooms                           4345 non-null   int64
5   neighborhood                        4345 non-null   object
6   status                              4326 non-null   object
7   newDevelopment                      4345 non-null   bool
8   parkingSpace                       2062 non-null   object
9   detailedType                       4345 non-null   object
10  floor                               3520 non-null   object
11  exterior                           3318 non-null   object
12  hasLift                            3570 non-null   object
13  newDevelopmentFinished              172 non-null    object
dtypes: bool(1), float64(2), int64(2), object(9)
memory usage: 445.7+ KB
```

Figure 1: Available columns in dataset after deleting redundant columns

3.1.3 Processing and formatting by column

3.1.3.1 Parking

Because initially, we have a boolean column indicating whether the property has parking or not, and a second column indicating the parking price. Since the only relevant information is the parking price we will sum the parking price to the total property price and we can remove the parkingSpace column now.

3.1.3.2 DetailedType

DetailedType indicates the type and subtype of each property. Seeing that 71,9% of the listings are flats we can just categorize the rest of the house typologies and subtypologies in the following 6 classes:

- penthouses
- IndependentHouses (we're adding all the chalets without subTypology assuming those are all independentHouses)
- terracedHouse (we group all the countryhouses to this class since there are just 10 values)
- semidetached house
- studio
- duplex

3.1.3.3 Status and newDevelopment

The columns 'status', 'newDevelopment' and 'newDevelopmentFinished' can provide us a lot of insight, first noting down that the newDevelopmentFinished is highly unbalanced with only 5 values set to true, so it will probably not provide much information.

We can observe that the amount of newly developed properties from the column 'status' doesn't fit with the ones from the column 'newDevelopment', coincidentally the difference between these two is the same as the invalid value count of the column 'status' so most probably this is the case but still we'll take a look at the faulty rows.

3.1.4 Ensuring Data Consistency

We process the column floor, first of all we should turn all of the values of this column into a numerical field:

- bj(bajo) was converted to 0.
- 'en' represents 'Entrepiso' and was treated as a missing value.

We also observe a significant imbalance in the floor data, with lower floors being more common. As the floor number increases, the number of data points decreases. The best solution for this is to group the floors above 10 into a single category.

Additionally, it's likely that the "floor" field has been used in two different ways: to describe the number of floors in a house and to indicate the specific floor number for apartments. We should investigate this further to ensure accurate interpretation.

In total, we have identified 825 missing values. These missing values are likely due to detached houses or villas, which may not have a floor specification. However, there could also be apartments without a specified floor. To address this, we will need to devise a meaningful strategy for handling these rows. For now, we propose grouping the missing values by home type and using the mode to fill in the gaps.

```

floor
1      625
4      520
2      481
3      465
bj      315
5      300
6      193
7      184
8      134
9      125
en       58
10       30
14       18
13       15
12       12
17       10
11        6
18        5
15        5
16        5
22        4
20        3
31        3
-1        2
21        1
34        1
Name: count, dtype: int64

```

Figure 2: Column floor

hasLift column:

- All houses had the field hasLift set to NaN
- Treated missing values as “no elevator”

```

hasLift
True      2567
False     854
Name: count, dtype: int64
Missing values for 'hasLift': 924
Houses with hasLift set to NaN: 8

```

Figure 3: Values of hasLift

3.2. Exploratory Data Analysis (EDA)

Exploratory Data Analysis is a critical step where we analyze the dataset to uncover patterns, correlations and potential issues. This phase helps us gain a deeper understanding of the data's structure and quality guiding subsequent preprocessing and modeling decisions. Through visualizations and sum

many statistics, we identify trends, detect outliers and ensure the dataset aligns with the project's goals.

3.2.1 Neighborhood

We analyze how property prices vary by neighborhood and provide us meaningful insights into the distribution and trends. By identifying neighborhoods with higher and lower average prices in this city, we can gain a deeper understanding of market dynamics and patterns.

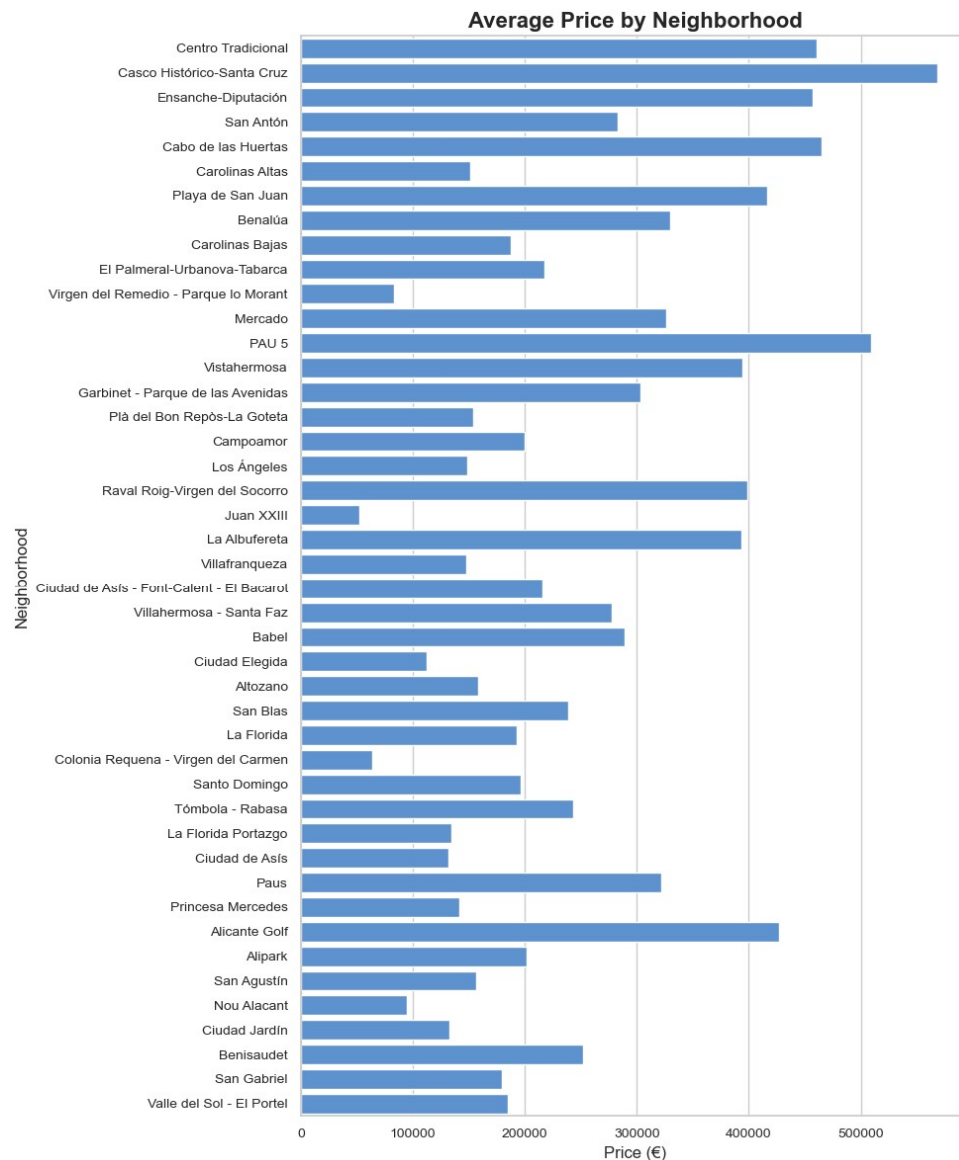


Figure 4: Average Price by Neighborhood

Categorical variables need to be converted to a numeric format so that machine learning models can handle them properly. One-Hot Encoding is a technique that transforms these variables into binary columns.

3.2.3 Outlier Handling for Numerical Columns

Outliers are extreme values in a database that deviate significantly from the majority of the data. These values can distort statistical analyses and negatively impact the performance of predictive models. For that reason we implemented a systematic approach to identify and treat outliers in the numerical columns.

To illustrate the effectiveness of this treatment, we provide a comparison of the numerical columns before and after outlier handling. This comparison demonstrates the reduced variability and improved distribution of values, which improves the stability and reliability of model training and analysis.

3.2.3.1 Distribution of flat prices

As we can observe in the first graph of figure 7, before the outlier treatment, properties with prices above 500,000€ and even exceeding 1,000,000€ are significantly reduced, reaching values above 2,000,000€. This can severely distort our predictions.

- We set a limit of 800,000€ and removed properties priced above this limit.

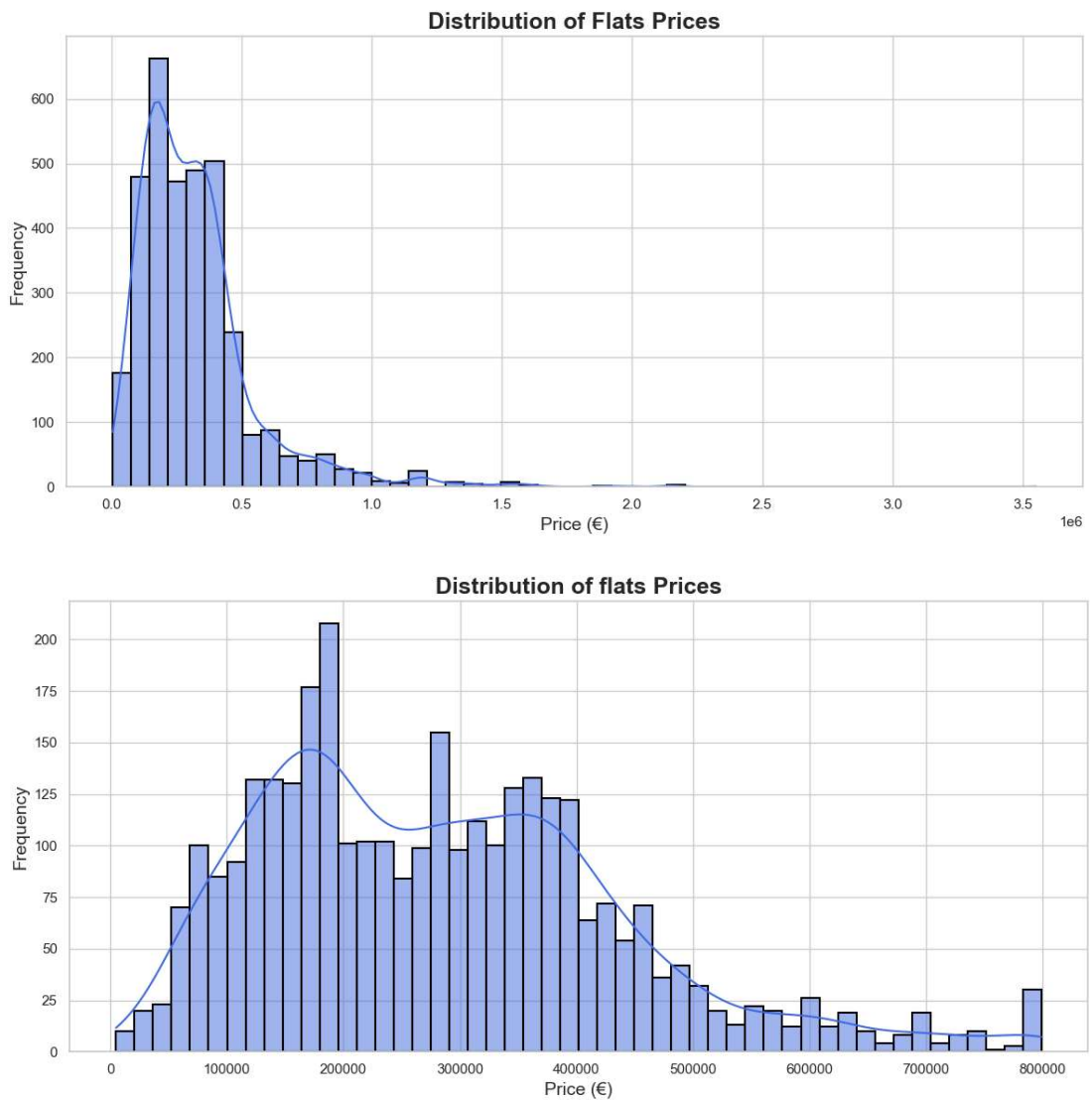


Figure 7: Comparative before and after outlier treatment. Distribution of flats prices.

3.2.3.2 Distribution of flat sizes

In figure 8 we can observe that the frequency of flats above 200 m² decreases significantly, and above 400 m², practically disappears. Therefore, a limit is set:

- We set a limit of 230 m²

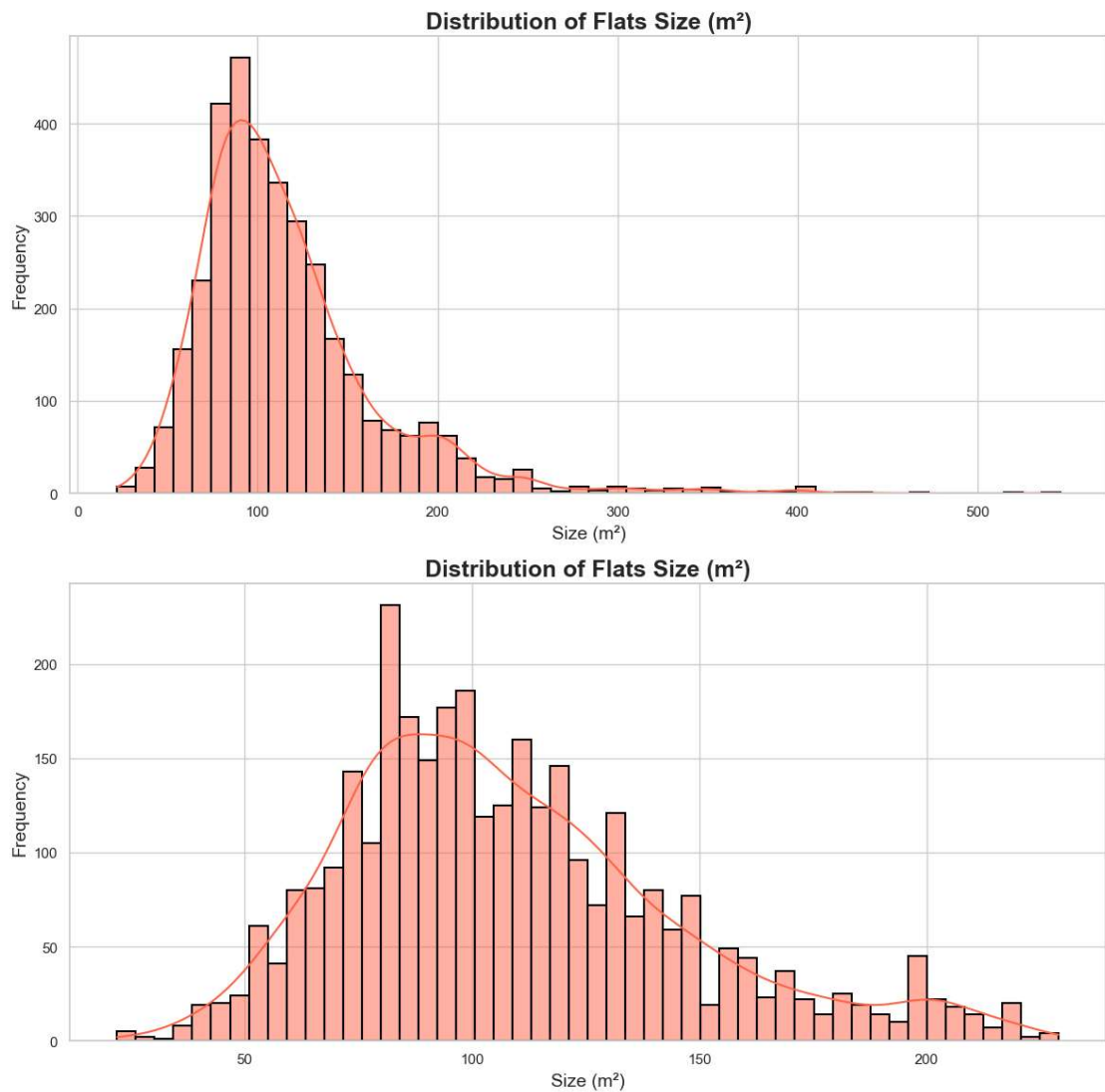


Figure 8: Comparative before and after outlier treatment. Distribution of flats Size (m2).

3.2.3.3 Distribution of flat rooms

As we can see in figure 9, the number of rooms in some properties is extraordinarily high, because these properties may represent not just a single apartment but an entire building or multiple properties sold together. To address this:

- We set a limit of 6 rooms, which we consider the maximum reasonable number for a single flat.

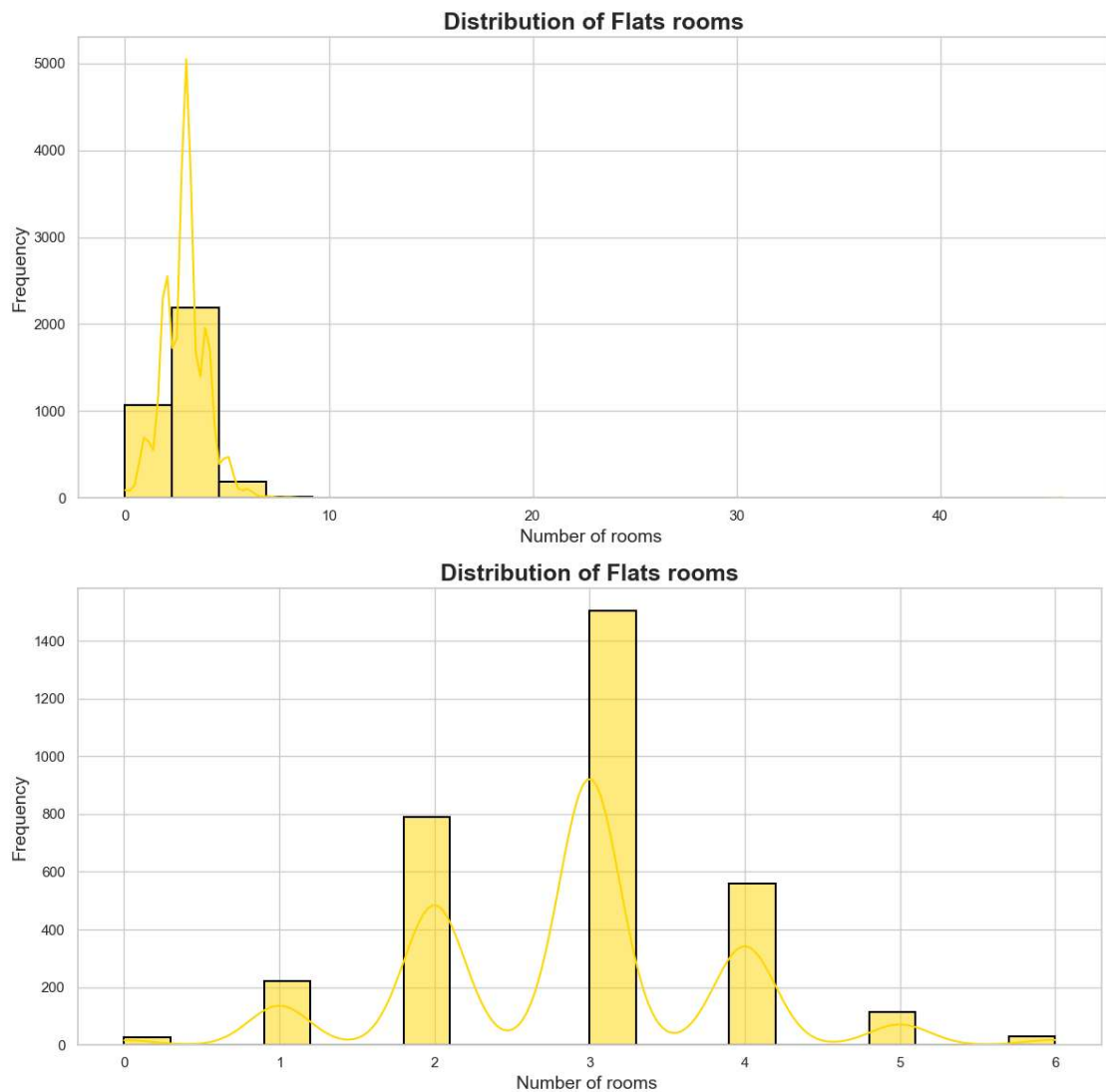


Figure 9: Comparative before and after outlier treatment. Distribution of flats rooms.

3.2.4 Visualization of Variable Interactions

3.2.4.1 Matrix of Relationships Between variables

This matrix allows us to observe how the numerical variables interact with each other, helping to identify patterns, correlations, distributions and ensuring that we have appropriately handled any anomalies and outliers.

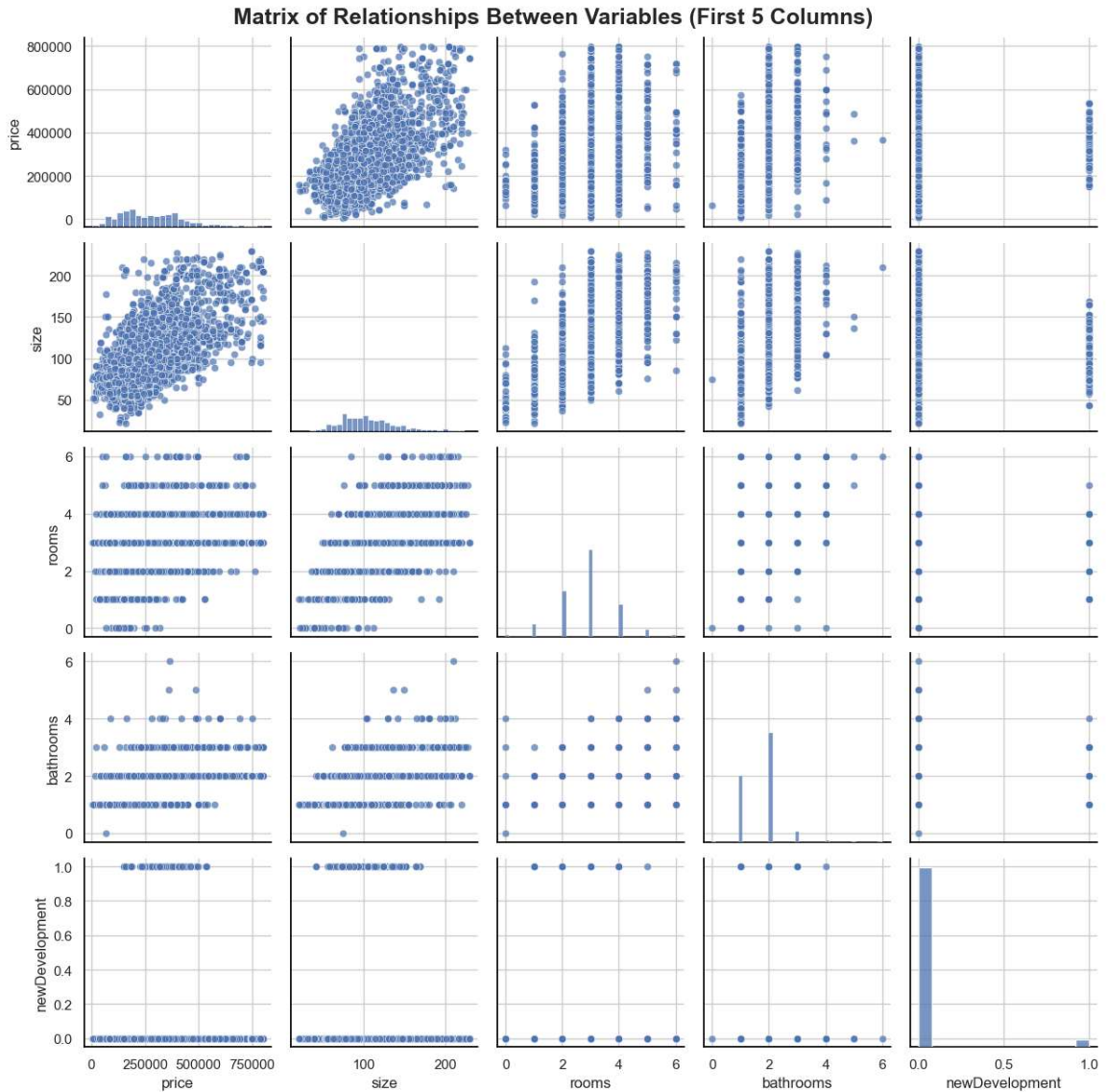


Figure 10: Matrix of Relationships Between variables (First 5 Columns)

3.2.4.1 Correlation heatmap

The heatmap helps visualize the relationships between numerical variables, showing which variables are positively or negatively correlated with each other.

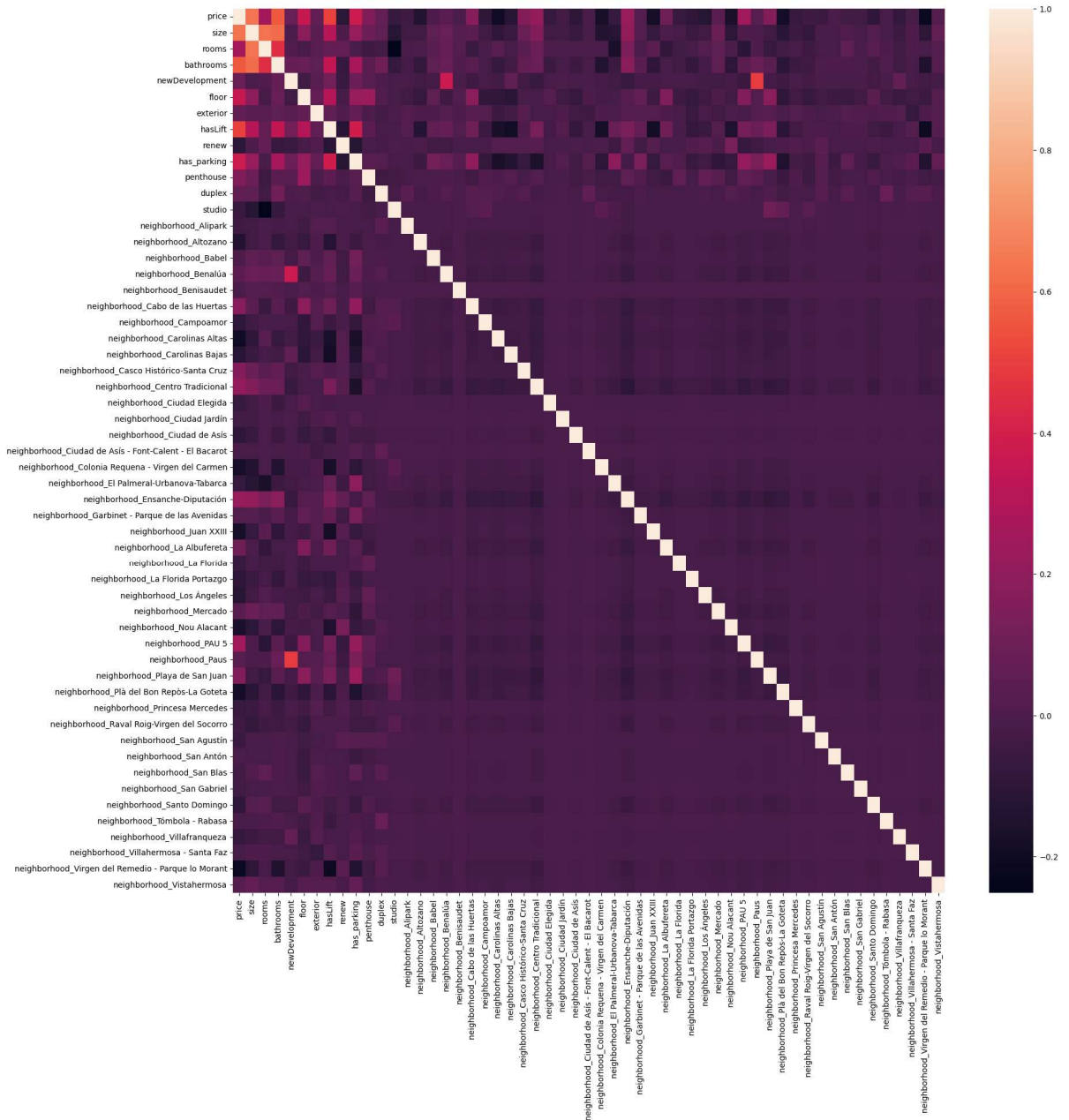


Figure 11: Correlation heatmap

3.3. Modeling

For this project we explored a wide range of regression models, in general, each model was fine-tuned using GridSearchCV, ensuring cross-validation to avoid sample bias and to find the best combination of hyperparameters. The scoring function chosen to optimize is 'neg_mean_squared_error' since we're interested in minimizing the error between the prediction of the model and the observed flat prices.

3.3.1 Ridge Regressor

The Ridge Regressor is a type of linear regression that incorporates regularization to prevent overfitting and improve generalization. This is achieved by adding an L2 penalty term to the cost function that shrinks the coefficients of less significant features.

This was the obvious place to start, since it's very efficient and interpretable, making it a great benchmarking model.

The only hyperparameter needed to be tuned is `param_ridge_alpha`, and we can observe on figure 12 that the scoring of the model worsens with higher values of alpha, so we used 1.

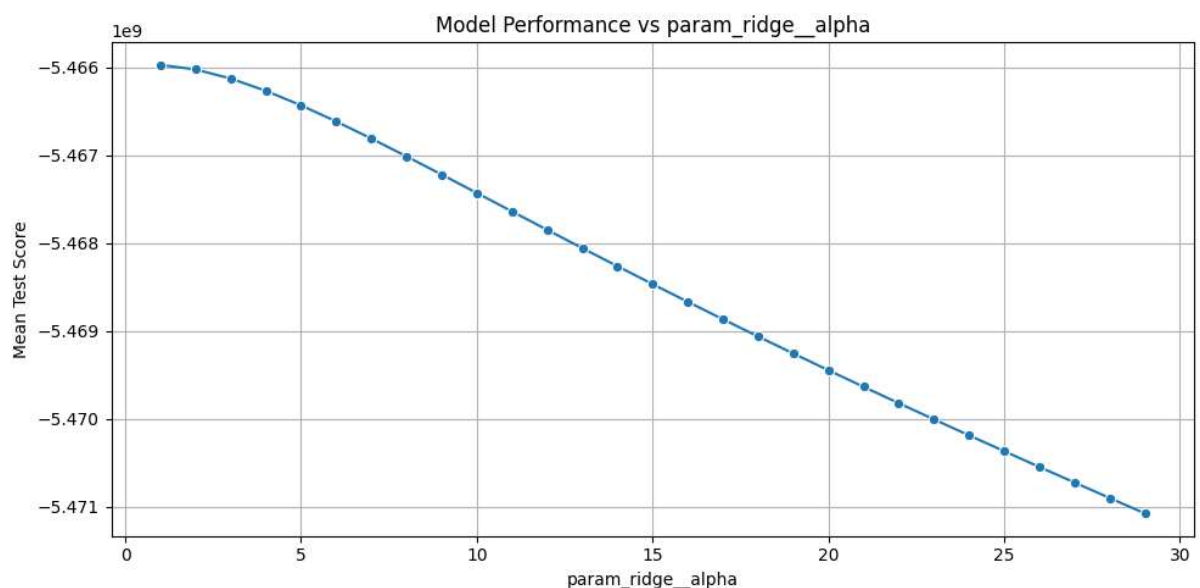


Figure 12: Alpha parameter tuning of the ridge regressor

Finally, in figure 13 the coefficients of the Ridge regressor are plotted. We can observe that in general, they align with common sense, giving positive values to features like size, bathroom s, newDevelopment, floor, penthouse, and recognizing some of the pricier neighbourhoods like Playa de San Juan and Casco Histórico Santa Cruz.

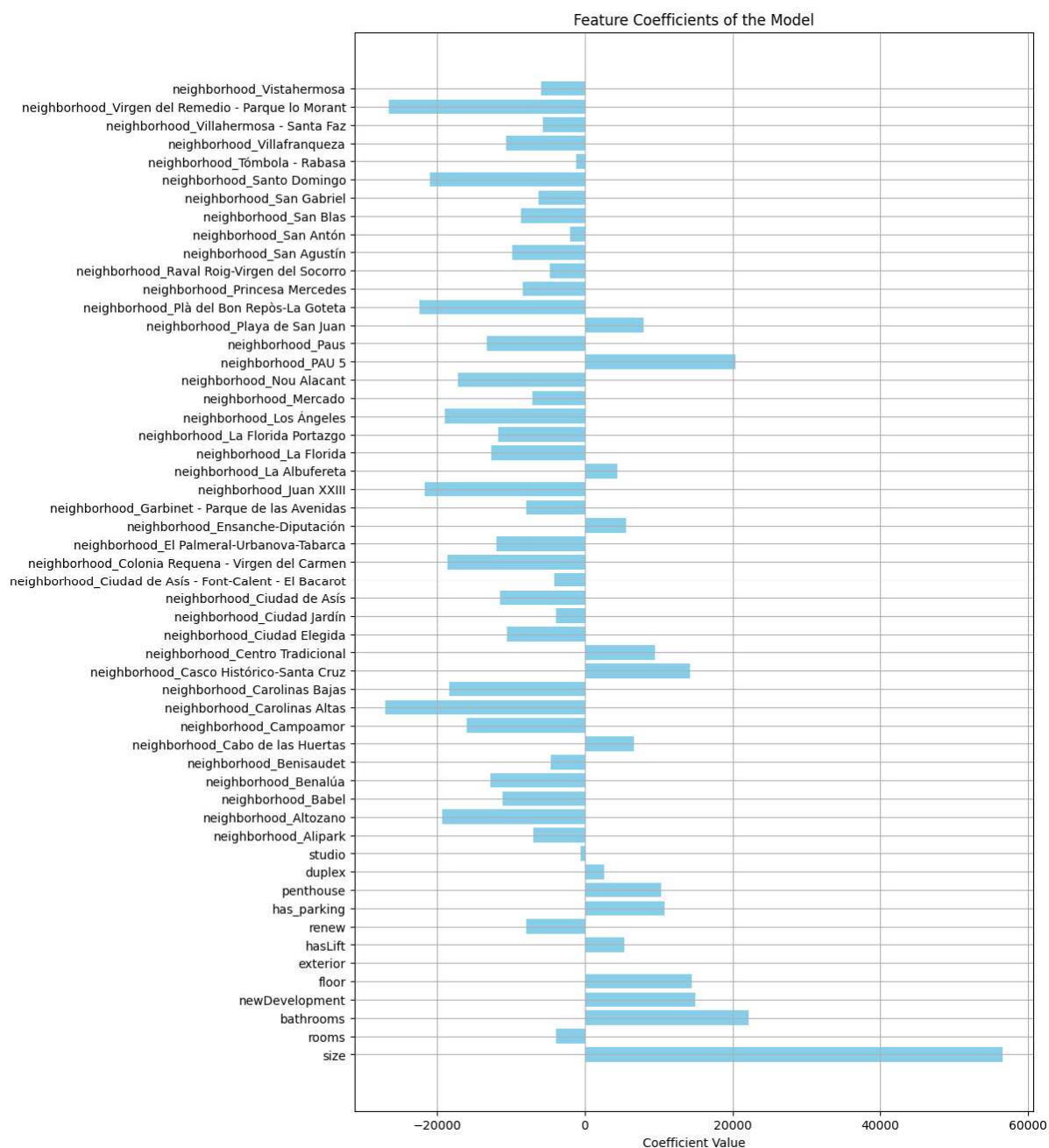


Figure 13: Coefficients of the Ridge regressor

3.3.2 Random Forest

The Random Forest is an ensemble method that uses multiple decision tree regressors on random sub-samples of the data, averaging their outputs to enhance accuracy and reduce overfitting.

We initially performed hyperparameter tuning using RandomizedSearchCV to explore a broad parameter space. This was followed by GridSearchCV to refine the parameters around the best results. Key parameters tuned included:

- `n_estimators`: number of trees
- `max_depth`: maximum depth of the trees
- `min_samples_leaf`: minimum samples required at a leaf node
- `min_samples_split`: minimum samples required to split a node

Balancing these parameters was crucial to manage model complexity, runtime, and the risk of overfitting.

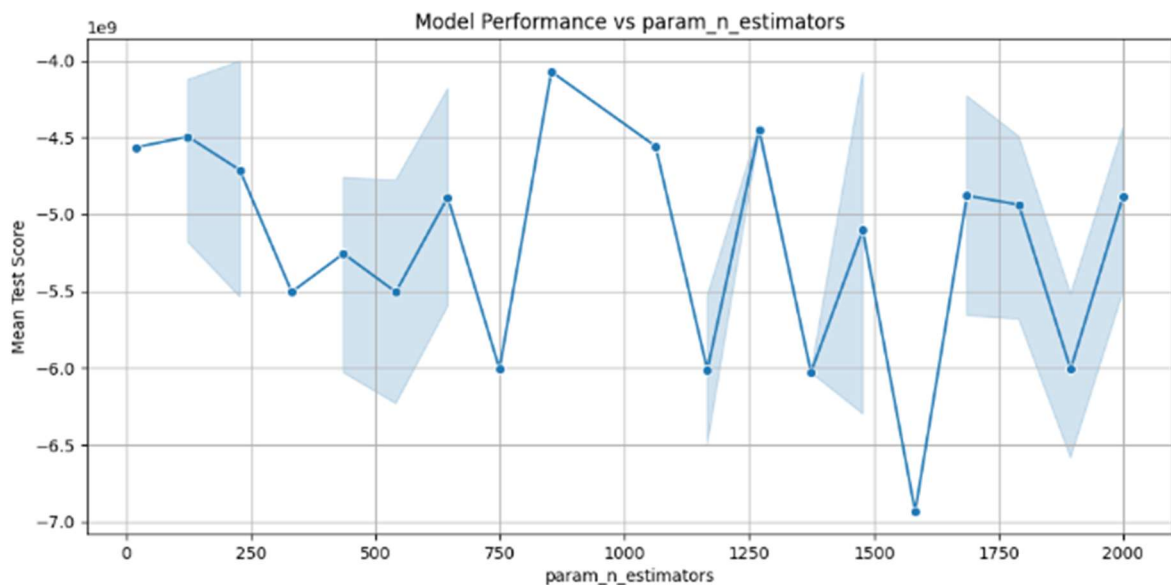


Figure 14: Evolution of the `n_estimators` parameter for the random forest regressor during the random search

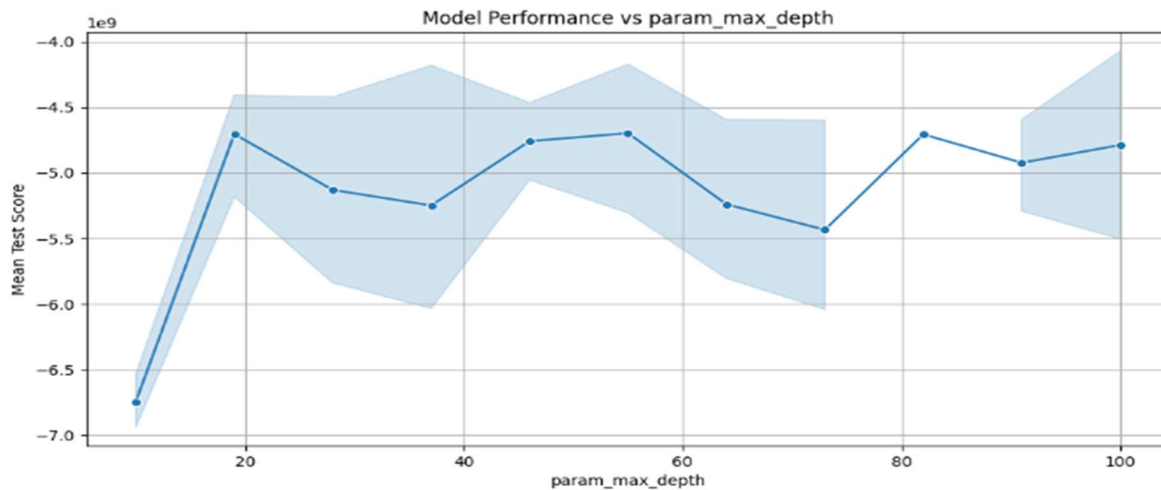


Figure 15: Evolution of the `max_depth` parameter for the random forest regressor during the random search

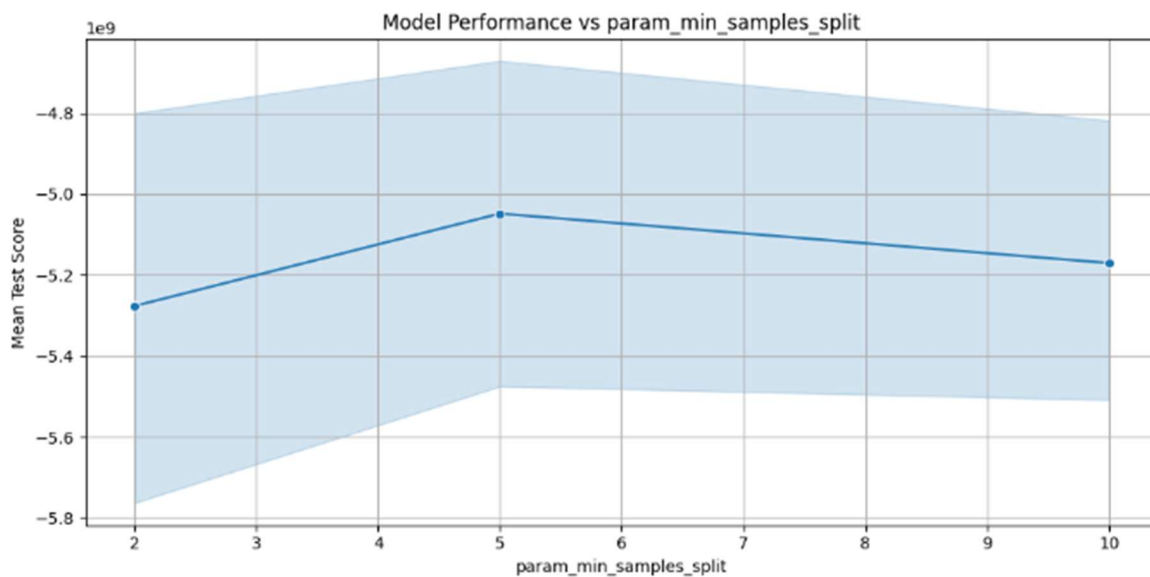


Figure 16: Evolution of the `min_samples_split` parameter for the random forest regressor during the random search

Since the best value for `n_estimators` is 781 on the random search, we'll just explore `n_estimators` and `param_max_depth`, to minimize the size of the `param_grid` and the training time.

3.3.3 Support Vector Regressor

Support Vector Regression (SVR) applies the principles of Support Vector Machines by mapping input features into a high-dimensional space to determine the optimal hyperplane that best fits the data.

For this dataset, the linear kernel was initially expected to perform best, but we also tested other kernel configurations, including sigmoid and RBF, using GridSearchCV to evaluate their effectiveness.

The main hyperparameters tuned were C and epsilon. The C value controls the trade-off between training error and the margin, influencing how penalties are applied to misclassified points. The epsilon parameter defines the margin of tolerance for errors, affecting the width of the tube around the hyperplane where no penalty is applied.

Despite these efforts, SVR showed poor performance on the training data, leading us to discontinue further exploration of this model.

3.3.4 Bagging regressor with Random Forest.

Bagging (Bootstrap Aggregating) is an ensemble technique that trains multiple base estimators on random subsets of the data and aggregates their predictions to reduce overfitting and variance.

In this case, we used the Random Forest as the base estimator and tuned only the `n_estimators` parameter for optimization, but we didn't even improve the performance of just a single random forest.

3.3.5 Gradient Boosting Regressor

GBR is a technique based on boosting where the target is pseudo-residuals instead of residuals as in traditional boosting. It gives a prediction in the form of an ensemble of weak prediction models, which are typically simple decision trees. It usually outperforms Random Forest.

Just like with the Random Forest we first performed a broad random parameter search to then do a more precise grid search to find the local minimum.

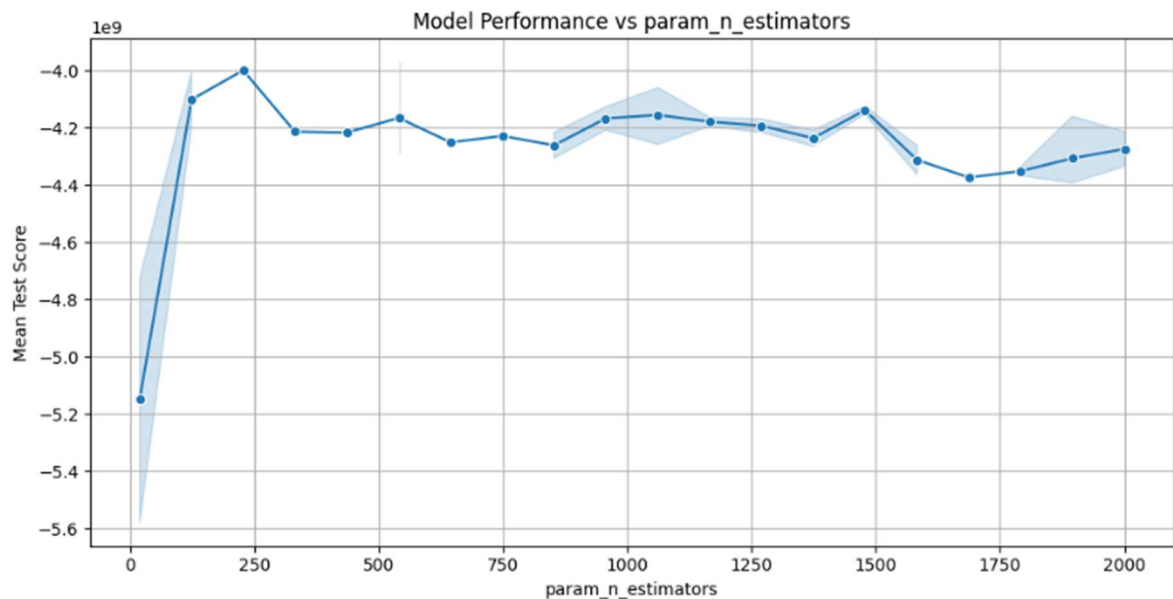


Figure 17: Evolution of the `n_estimators` parameter for the GBR during the random search

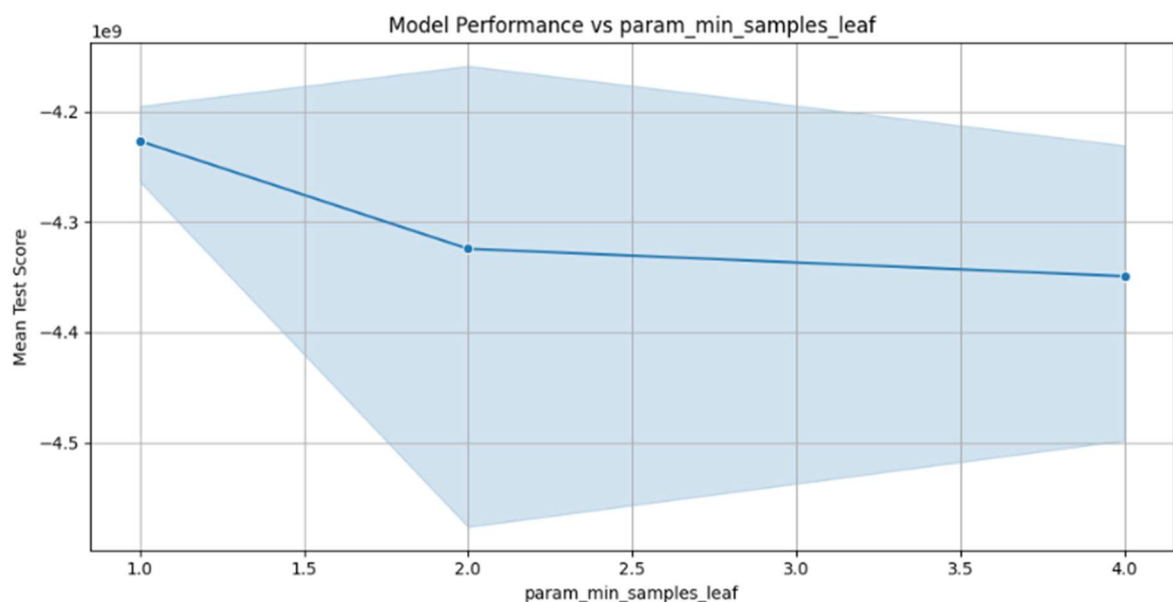


Figure 18: Evolution of the `min_samples_leaf` parameter for the GBR during the random search



Figure 19: Evolution of the max_depth parameter for the GBR during the random search

The best parameters of the random search were:

- n_estimators: 541
- min_samples_split: 5
- min_samples_leaf: 4
- max_features: sqrt
- max_depth: 10

Further fine-tuning around these values using GridSearchCV did not yield better results.

3.3.6 XGBoost Regressor

XGBoost is an efficient implementation of gradient boosting with additional regularization techniques to prevent overfitting.

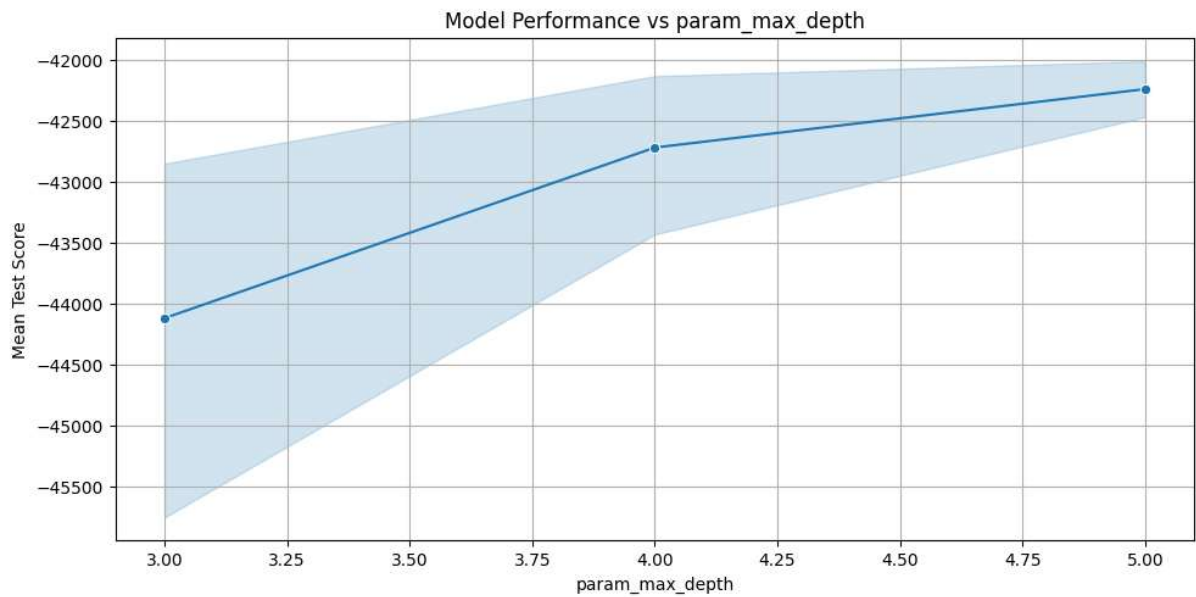


Figure 20: Evolution of the max_depth parameter for the XGBR

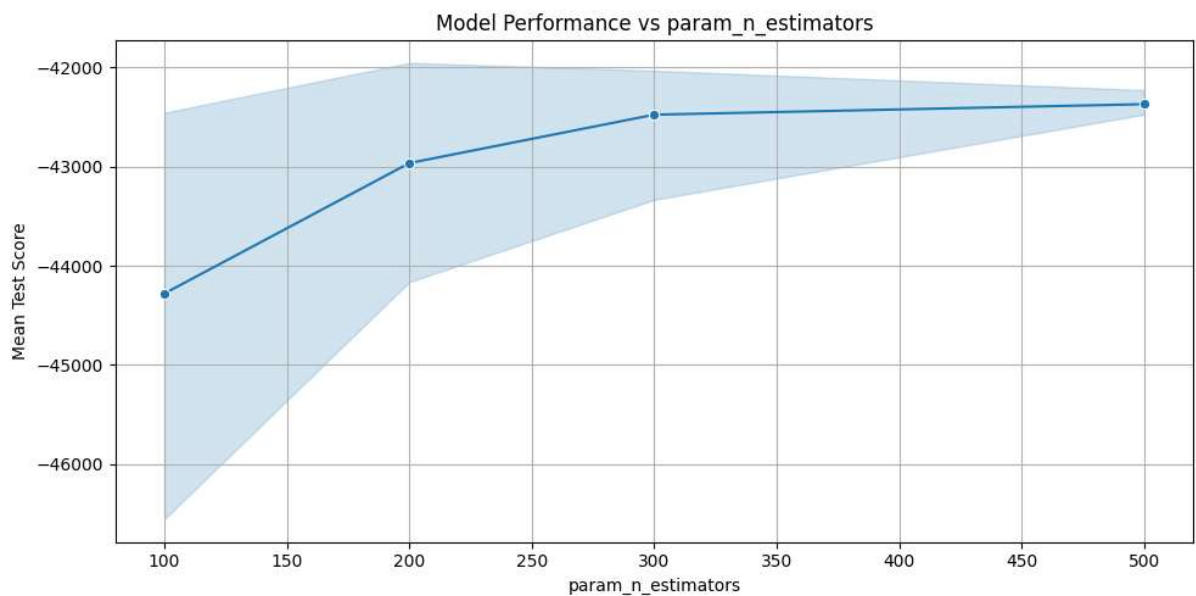


Figure 21: Evolution of the n_estimators parameter for the XGBR

3.3.7 Light Gradient Boosting Regressor

LightGBM is a gradient boosting framework that uses tree-based learning algorithms and is designed for efficiency and scalability.

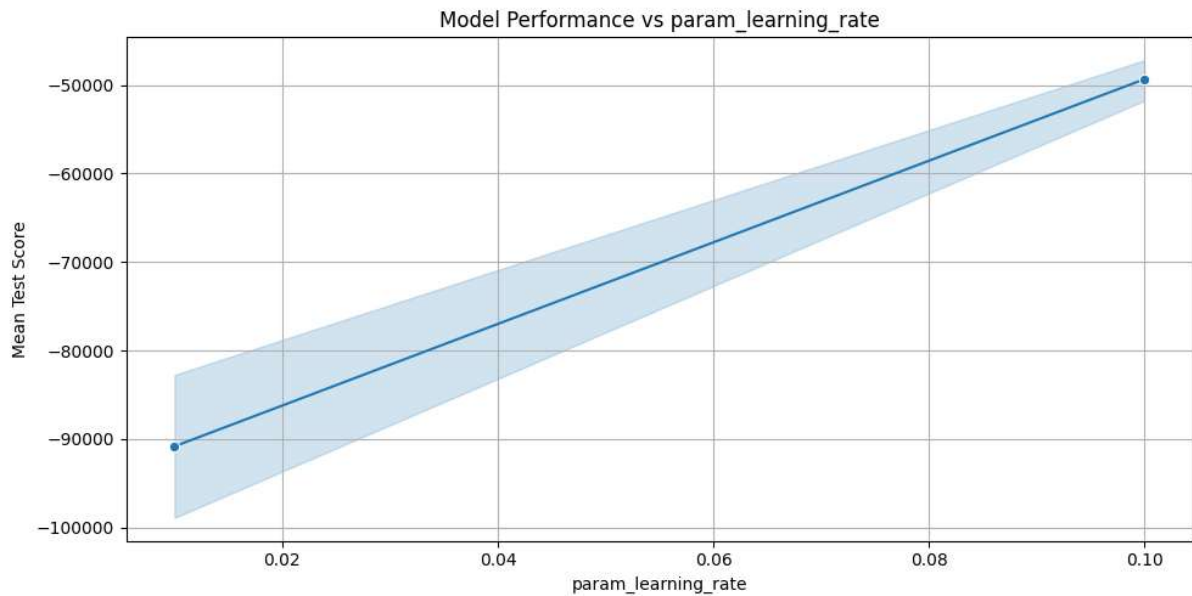


Figure 22: Evolution of the learning rate parameter for the LGBR

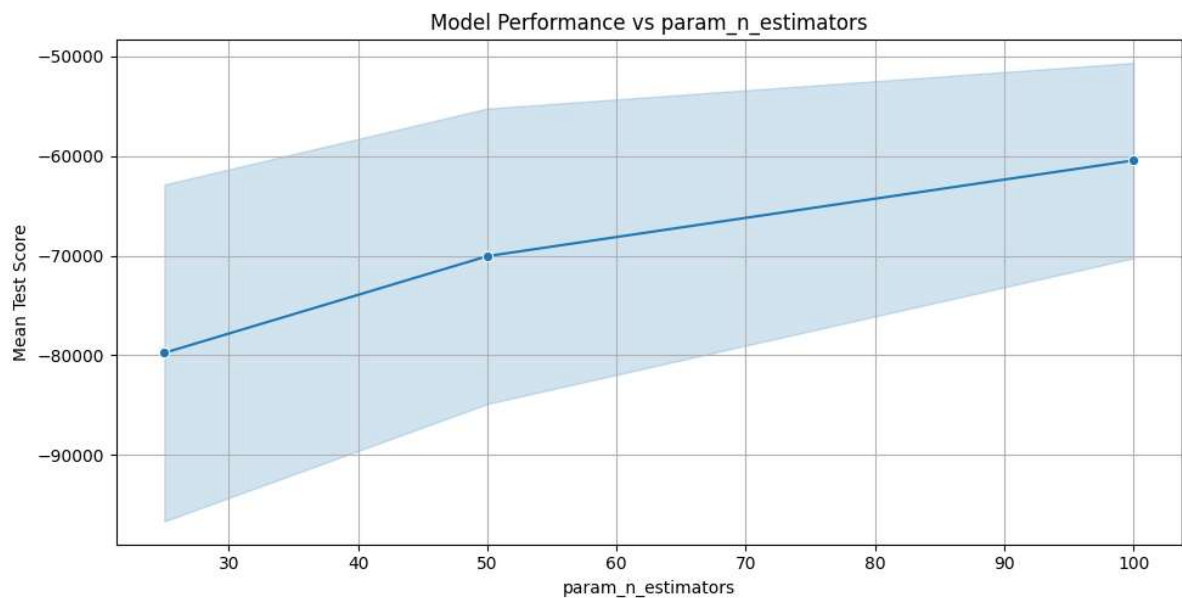


Figure 23: Evolution of the n_estimators parameter for the LGBR

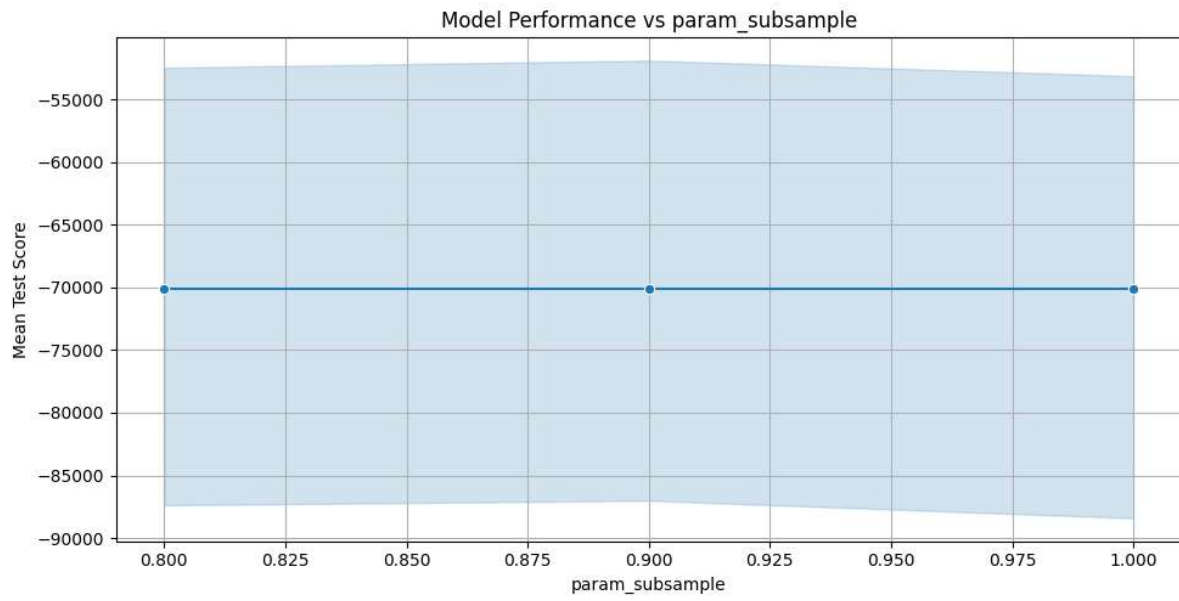


Figure 24: Evolution of the subsample parameter for the LGBR

3.3.8 Cat Boosting Regressor

Cat Boosting (Categorical Boosting), is a high-performance, open-source, gradient-boosting framework. It is designed for solving a wide range of machine learning tasks, including classification, regression, and ranking, with a particular emphasis on handling categorical features efficiently. Catboost stands out for its speed, accuracy, and ease of use in dealing with structured data.

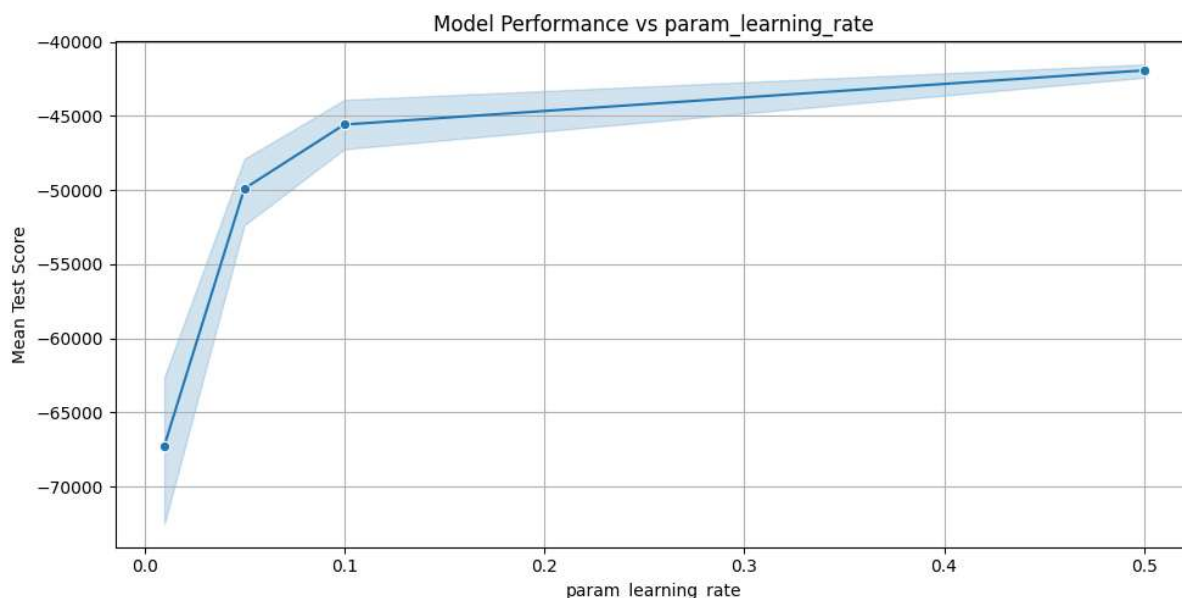


Figure 25: Evolution of the learning rate parameter for the Cat Boosting Regressor

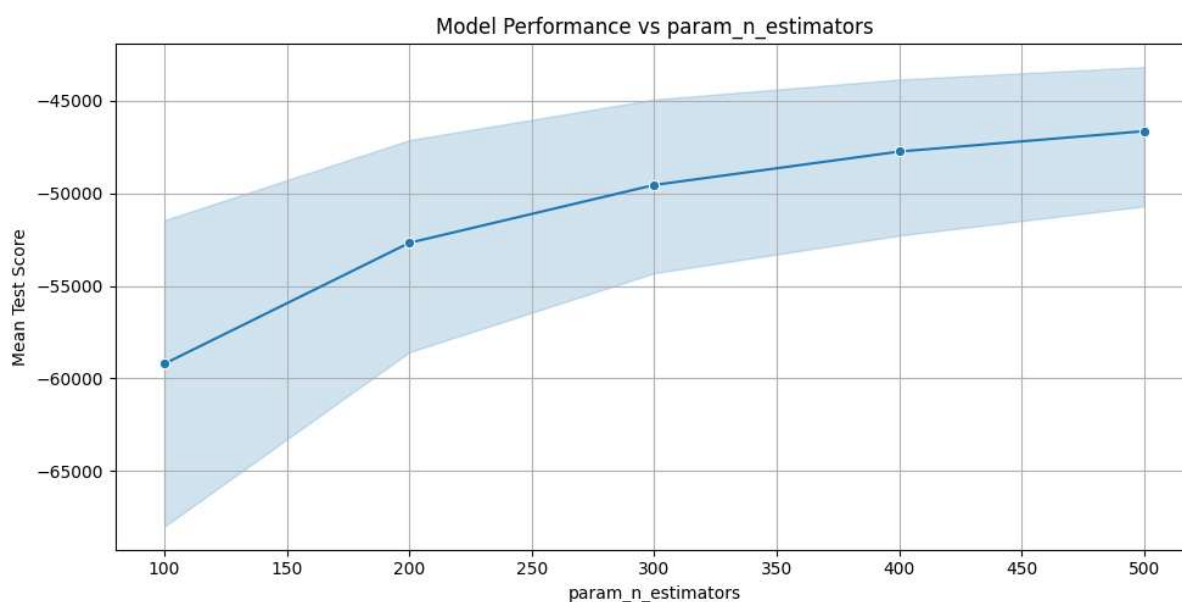


Figure 26: Evolution of the $n_estimators$ parameter for the Cat Boosting Regressor

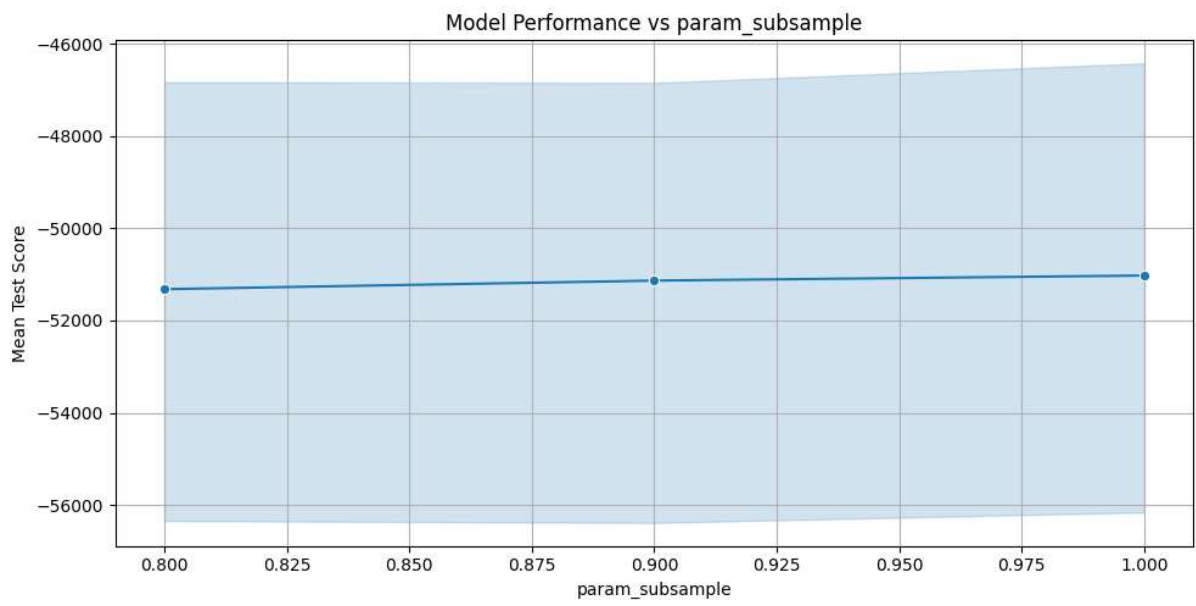


Figure 27: Evolution of the subsample parameter for the Cat Boosting Regressor

3.4. User Interface

This project has mainly focused on the development of the core artificial intelligence, specifically in the creation and training of the housing price prediction model. The main objective has been to build an accurate model to predict the price of properties based on factors such as size, number of rooms, location and other attributes. The user interface that would allow end users to interact with the prediction model hasn't been implemented but it could be easily done using frameworks like Streamlit or Flask.

3.5. Testing and Improvements

The dataset was split into an 80-20 proportion for training and testing, ensuring that the testing subset was not used during model training. Cross-validation was employed during the training process to further improve the robustness of the models.

The results for each model on the testing subset are summarized below:

Model	RMSE	R ²
GBR	66,628.3293	0.8319
Random Forest	69,138.8062	0.8190
CatBoosting	69,406.9369	0.8176
Bagging	70,585.0012	0.8114
XGBR	72,719.6301	0.7998
Ridge	81,567.3977	0.7481
LGBM	105,587.3743	0.5779
SVR	118,173.1621	0.4713

Table 1: Results of each model on testing subset.

We can observe that the Gradient Boosting Regressor (GBR) outperformed all other models with the lowest Root-mean-square deviation (RMSE) of 66,628.33 and the highest R² score of 0.8319. It was closely followed by Random Forest and CatBoosting, which also demonstrated strong performance. In contrast, models such as LGBM and SVR performed less effectively, indicating potential limitations in their parameter tuning or suitability for this dataset.

Due to time constraints, we could not conduct exhaustive parameter tuning for some models, particularly LGBM, XGBoost, and CatBoost. Additionally, further exploration of alternative ensemble methods and advanced optimization techniques may lead to improved results. These approaches could include fine-tuning hyperparameters, experimenting with stacking or blending ensemble methods, or applying domain-specific feature engineering to enhance predictive performance.

Overall, the results suggest that boosting methods like GBR and CatBoosting are highly effective for this dataset, and further refinements could strengthen the outcomes even further.

4. Projected Impact

4.1. Accomplishments and Benefits

4.1.1 Initial Goals Achieved

The primary objectives of the project have been successfully met. The aim was to conduct a price prediction study focused on a specific city, in this case, Alicante.

4.1.2 Collaboration with Idealista

We successfully established contact with Spain's largest real estate platform, *Idealista*. Through access to their free API tier, we were able to retrieve real-time data for all current property listings in Alicante.

4.1.3 Data Preprocessing

The data obtained via JSON API calls were preprocessed effectively to create a structured and meaningful dataset of property information.

4.1.4 Feature Engineering and Transformation

Independent variables were carefully analyzed, and those deemed irrelevant for training were removed. Additionally, categorical data were transformed into numerical formats using techniques like *one-hot encoding*, applying the knowledge gained throughout the course.

4.1.5 Algorithm Implementation and Evaluation

Multiple algorithms were implemented to evaluate predictive performance. Advanced ensemble techniques, such as *bagging* and *boosting*, were also applied to enhance model accuracy and robustness.

4.1.6 Graphical Analysis and Visualization

Data visualization libraries were employed to generate clear and insightful graphical analyses, facilitating better interpretation and communication of results.

4.1.7 Educational and Practical Value

The project delivered evident benefits both academically and practically. It provided hands-on experience with real-world challenges associated with working on actual datasets, surpassing the scope of examples typically encountered in training courses.

These accomplishments not only reflect technical success but also highlight the project's potential for real-world applications in the real estate sector.

4.2. Future Improvements

4.2.1 Enhancement of the Training Dataset

One of the primary improvements involves expanding the training dataset. Currently focused on the city of Alicante, the dataset should be scaled to encompass data from across Spain. Achieving this would likely require establishing agreements with the owners of such datasets, enabling access to more comprehensive and representative data.

4.2.2 Elimination of Outdated Data

During the training process, it was observed that some properties have been listed for significantly longer periods than others, distorting the training outcomes. With a larger and more robust dataset, it would be feasible to either eliminate these outdated data points or adjust them to better reflect current market realities.

4.2.3 Integration of Points of Interest

Analysis revealed notable price divergences within the same neighborhood, often influenced by proximity to key locations such as beaches, historical centers, or commercial areas, even if the distances are relatively short. Including points of interest and their distances from properties in the dataset would further enhance predictive accuracy.

4.2.4 Inclusion of Property Age Data

Another critical improvement would be incorporating the age of the properties. Despite its significant influence on property valuation, this information was not available in the current dataset. Including this variable would add another layer of precision to the predictions.

4.2.5 Access to Real Selling Prices

A key limitation was the reliance on listed prices rather than the actual selling prices of properties. Often, the final transaction value differs from the asking price. Access to historical sales data from real estate agencies would greatly enhance the model's accuracy and reliability.

4.2.6 Scalable Algorithm Distribution

The scalability of the algorithm would benefit immensely from collaboration with intermediary sales platforms and real estate agencies. Their integration would not only expand the dataset further but also improve the algorithm's real-world applicability and market adoption.

These enhancements, if implemented, would position the project as a cutting-edge tool for real estate price prediction, offering unparalleled value to both property owners and market analysts.

5. Team Member Review and Comment

--

NAME	REVIEW and COMMENT

6. Instructor Review and Comment

CATEGORY	SCORE	REVIEW and COMMENT
IDEA	10/10	
APPLICATION	30/30	
RESULT	30/30	
PROJECT MANAGEMENT	10/10	
PRESENTATION & REPORT	20/20	
TOTAL	100/100	